

separated by a comma: the format is `[row, column]`. For example, if we wanted the row with index 4 and the column with index 1, we could write `half_double.loc[4,1]`. In this case, we will do more than just specify indices. We will express a logical pattern for which rows we want: we want all rows where `halving` is odd. We specify the `halving` column in our logic with `half_double[0]`, since it's the column with index 0. We specify oddness with `%2 == 1`. Finally, we specify that we want all columns after the comma by writing a colon, which is a shortcut indicating that we want every column.

Finally, we simply take the sum of the remaining doubling entries:

```
answer = sum(half_double.loc[:,1])
```

Here, we are using `loc` again. We specify inside the square brackets that we want every row by using the colon shortcut. We specify that we want `doubling`, the column with index 1, after the comma. Note that the 89×18 example we worked through could be done more quickly and easily if we instead calculated 18×89 —that is, if we put 18 in the `halving` column and 89 in the `doubling` column. I encourage you to try this to see the improvement. In general, RPM is faster if the smaller multiplicand is placed in the `halving` column and the larger one in the `doubling` column.

To someone who has already memorized the multiplication table, RPM may seem pointless. But besides its historical charm, RPM is worth learning for a few reasons. First, it shows that even something as dry as multiplying numbers can be done in multiple ways and is amenable to creative approaches. Just because you've learned one algorithm for something doesn't mean that it's the only, or the best, algorithm for the purpose—keep your mind open to new and potentially better ways of doing things.

RPM may be slow, but it requires less memorization up front because it doesn't require knowledge of most of the multiplication table. Sometimes it can be very useful to sacrifice a little speed for the sake of low memory requirements, and this speed/memory tradeoff is an important consideration in many situations where we're designing and implementing algorithms.

Like many of the best algorithms, RPM also brings into focus relationships between apparently disparate ideas. Binary expansions may seem like just a curiosity, of interest to transistor engineers but not useful to a layperson or even a professional programmer. But RPM shows a deep